SA ID Number Validation — Technical Reference

PhilaniFinance | Source: `src/lib/saIdValidator.ts` | Generated: 25 May 2026

Overview

The `validateSaId(id: string)` function validates South African identity numbers against the official 13-digit format. It returns `{ valid: boolean, message: string }` and applies four sequential checks. Validation fires live in the application form as soon as the user types a 13-digit value.

Validation Steps

1 Format Check

The entire string must match the regular expression `/^\d{13}$/` — exactly 13 numeric characters, no spaces or letters.

```
if (!/^\d{13}$/.test(id)) {
  return { valid: false, message: 'ID number must be exactly 13 digits.' };
}
```

2 Date of Birth Extraction & Validation

The first 6 digits encode the holder's birth date: `YYMMDD`.

- Digits `0–1` → year (2-digit)
- Digits `2–3` → month (must be 1–12)
- Digits `4–5` → day (must be 1–31)

```
const year  = parseInt(id.substring(0, 2), 10);
const month = parseInt(id.substring(2, 4), 10);
const day   = parseInt(id.substring(4, 6), 10);

if (month < 1 || month > 12) { /* invalid month */ }
if (day   < 1 || day   > 31) { /* invalid day   */ }
```

The full year is inferred: if the 2-digit year ≤ current 2-digit year → 2000s, else 1900s.

3 Citizenship Digit

Digit at position `10` (0-indexed) must be `0` or `1`.

- `0` — South African citizen
- `1` — Permanent resident

```
const citizenship = parseInt(id[10], 10);
if (citizenship !== 0 && citizenship !== 1) {
```

```
  return { valid: false, message: 'ID number has an invalid citizenship digit.' };
}
```

4 Luhn Checksum

The 13th digit is a check digit computed via the **Luhn algorithm** over the first 12 digits:

- **Even-indexed positions** (0, 2, 4 …): add the digit as-is.
- **Odd-indexed positions** (1, 3, 5 …): double the digit; if the result > 9, subtract 9.
- Check digit = `(10 - (sum % 10)) % 10` — must equal digit 13.

```
let sum = 0;
for (let i = 0; i < 12; i++) {
  if (i % 2 === 0) {
    sum += digits[i];
  } else {
    const doubled = digits[i] * 2;
    sum += doubled > 9 ? doubled - 9 : doubled;
  }
}
const checkDigit = (10 - (sum % 10)) % 10;
if (checkDigit !== digits[12]) { /* Luhn check failed */ }
```

Where Validation Is Triggered

Trigger	Location	Behaviour
Live keystroke	`ApplicationForm.tsx` — `update()`	Runs as soon as input reaches 13 digits; shows inline error or success message.
Profile auto-fill	`ApplicationForm.tsx` — `useEffect`	Validates the stored ID when saved profile data is loaded into the form.
Step gating	`ApplicationForm.tsx` — `canProceed()`	Blocks the user from advancing past Step 1 unless the ID is valid.

Return Value

On success the message includes the decoded date of birth, e.g.:

```
// Valid
{ valid: true,  message: "Valid SA ID. Date of birth: 1/1/1980" }

// Invalid
{ valid: false, message: "ID number failed the Luhn checksum. Please check your ID number." }
```